IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 30 July 2025, accepted 20 August 2025, date of publication 1 September 2025, date of current version 8 September 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3604563

APPLIED RESEARCH

Enhancing Student Retention in Introductory Programming Courses: Leveraging Advanced Learning Validation Tools and Educational Data Mining

ALAN MUTKA, FATIMA ŽIVKOVIĆ MUTKA, MARTIN ŽAGAR, AND DOMAGOJ TOLIĆ
RIT Croatia, 10000 Zagreb, Croatia

Corresponding author: Alan Mutka (alan.mutka@croatia.rit.edu)

This work was supported by the Provost's Learning Innovation Grants, Rochester Institute of Technology.

ABSTRACT Student retention in introductory programming courses remains a persistent challenge in higher education, with high failure and dropout rates impacting both learners and institutions. This article presents a novel, behavior-based approach to addressing this issue through Educational Data Mining (EDM) and a custom-built learning validation framework called AssessMe. Developed by the authors in collaboration with SmoothSoft Ltd., AssessMe is an advanced software tool that monitors the real-time development of programming assignments. It captures detailed behavioral data–including active coding time, code changes (added, modified, removed lines), and submission timelines–to generate learning indicators reflecting how students approach problem-solving tasks. Unlike traditional assessment methods that focus solely on final code correctness, AssessMe emphasizes the coding process, offering deeper insights into student engagement, effort, and learning strategies. This study focuses on Programming I and II courses within the Web & Mobile Computing program at RIT Croatia. The dataset includes 3,537 student submissions from the 2024/2025 academic year, covering homework, practicals, and in-class activities, all enriched with AssessMe indicators. We apply traditional machine learning models combined with the TSFRESH library to extract meaningful time-series features from students' coding activity. This enables the identification of temporal learning patterns and supports early prediction of academic outcomes. Our models achieve over 93% accuracy in forecasting pass/fail status by the fifth week of a 15-week semester, demonstrating a strong correlation between AssessMe indicators and final grades. This behavior-based assessment approach enhances early intervention strategies and provides actionable insights for improving student retention and learning outcomes.

INDEX TERMS Educational data mining, machine learning, student retention.

I. INTRODUCTION

The landscape of higher education is marked by a persistent challenge – student retention. This issue is particularly pronounced in computer science, where introductory programming courses often serve as a pivotal hurdle for many students. The high dropout and failure rates associated with these courses have significant implications for both individual students and educational institutions. Students who struggle or drop out early in their academic journey may face diminished career prospects and reduced earning potential, while institutions suffer from decreased enrollment, lower graduation rates, and diminished reputation [1], [2].

A multitude of factors contribute to the high dropout rates observed in introductory courses. Academic challenges such as the steep learning curve associated with programming concepts can be overwhelming for many students, leading to frustration and discouragement. Lack of prior

The associate editor coordinating the review of this manuscript and approving it for publication was Alba Amato.

© 2025 The Authors. This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

programming experience can also be a significant barrier, as students without a solid foundation may struggle to grasp fundamental concepts and problem-solving skills. Additionally, the transition from high school to university can be challenging for many students, particularly those who are new to the academic environment. Furthermore, the literature suggests that students' ability to succeed is tied to the educators' expectations and the support structures provided to meet those expectations [3]. Institutional factors, such as inadequate advising and support services, can significantly contribute to student attrition. For instance, students may make poor decisions regarding course selection, major changes, or withdrawal from classes [4].

A high graduation rate is crucial for maintaining a robust student retention rate. However, achieving this goal can be challenging, as evidenced by the European Union's average of 9.7% ''early school leavers'' in 2021, according to Eurostat [5]. To tackle this issue, it is essential to identify students who are at risk of dropping out early [6], particularly freshmen during their first semester. Early intervention strategies can then be implemented to support these students and increase their likelihood of completing their education.

In computer science education, introductory programming courses often face high dropout and failure rates, with students withdrawing or failing at rates as high as 30-50% [7], [8]. These courses are typically challenging, as they establish the foundational knowledge necessary for all subsequent coursework. When students encounter negative experiences early in their academic journey, they are more likely to consider dropping out. To address this issue, it is crucial to implement detection systems that monitor at-risk students, understand the reasons behind their struggles, and develop preventive strategies. Early detection in the learning process allows for timely feedback and interventions, significantly increasing students' chances of successfully completing the course.

To effectively address the challenges faced by students in introductory programming courses and enhance retention rates, institutions must implement a multifaceted approach [9], [10]. Early intervention strategies, including but not limited to academic support services, personalized learning approaches, and targeted interventions for at-risk students, are essential. Academic support services can provide students with the resources they need to overcome academic challenges and stay on track. Personalized learning approaches, tailored to individual student needs and learning styles, can foster deeper engagement and understanding. Moreover, early identification of at-risk students through predictive analytics or other methods enables institutions to provide targeted support and interventions, preventing students from falling behind and increasing their chances of success.

Previous approaches for predicting course performance frequently depend on students' educational history and socioeconomic background to identify correlations with success rates. Alternatively, other methods use extensive tracking of student behavior, including their interactions, submission patterns, and engagement levels, to forecast outcomes. These approaches all utilize various data mining techniques [11], but the main differences lie in the importance of selecting the correct input data.

Authors [12], [13] aim to predict academic outcomes for freshman Computer Science students based solely on the academic data such as overall course grade and grades of specific course elements. In addition to academic data, the authors [14] use admission data, including date of birth, previous schools attended, and other past educational success metrics. Given our objective of early identification of at-risk students, the previously proposed approaches are not suitable for our purposes.

A. EDUCATIONAL DATA MINING

Educational Data Mining (EDM), as discussed in [11] and [15], is a specialized branch of data mining that focuses on extracting insights from academic databases. The goal of EDM is to analyze educational data to enhance our understanding of student learning behaviors and outcomes. This field leverages various platforms, including Learning Management Systems (LMS), Massive Open Online Courses (MOOCs), and other custom-developed educational technologies [16], [17], [18]. These platforms gather extensive, multi-modal data from student activities, tracking metrics such as how often and when students access learning resources, their performance on assignments, feedback from repeated submissions, and the time remaining before assignment deadlines. This data helps in identifying patterns and improving educational strategies.

The authors in [19] use TIM (The Interactive Material) [20], a multi-functional document-based learning management system, to extract features for predicting pass/fail outcomes. Key features include the number of correct submissions before deadlines, incomplete tasks, and timeliness of task completion. Iterative problem-solving behaviors, such as the number of incorrect answers and subsequent submissions, are also important. The prediction models, especially random forest and logistic regression, achieve balanced accuracy between 64% and 88%, with F1 scores ranging from 0.48 to 0.84. Similarly, the research [21] uses CodeBench, an online judge system, to analyze and predict student performance in programming courses. CodeBench collects detailed interaction data such as keystrokes, submissions, and code tests, which are used to create predictive models. These models, including decision trees and logistic regression, achieve an F1-score of about 80% and an accuracy of around 81% early in the course, improving to approximately 90% by the sixth week. Furthermore, the paper [22] employs CodeBench and INGInious, an automated source code evaluation tool, to analyze student performance in programming courses. Features used for prediction include grades, the number of attempts, delivery times, and interaction data from CodeBench. The models, using various classification algorithms such as decision trees, logistic regression, and

neural networks, achieve prediction accuracy ranging from 80% to 87%, with F1 scores exceeding 83% for some models.

Implementing the presented pass/fail prediction solutions in our existing courses requires using dedicated platforms with similar functionalities, which is often impractical as changing the LMS for only a few courses is not best practice. Most solutions also require auto-grading systems, necessitating the inclusion of all assignments on the platform with specially developed testers, demanding significant additional time. Additionally, the latest developments in LLMs (Large Language Models), such as ChatGPT, allow students to quickly generate solutions for their assignments, potentially undermining the assessment process. Moreover, addressing privacy and data security concerns involves the anonymization of data and complying with regulations to protect students' personal information. These platforms require careful planning and coordination among educators and IT professionals to balance transparency, privacy, and technical feasibility, adding to the complexity of implementation.

This article introduces an innovative assessment framework called **AssessMe**, designed to evaluate how programming assignments are created, rather than just assessing the final product. **AssessMe** is developed by Smoothsoft Ltd. in close collaboration with RIT Croatia, which serves as the academic testbed and evaluation environment. The system is built to ensure academic integrity and support learning validation within educational institutions by capturing fine-grained, real-time data on the coding process.

AssessMe tracks code development over time, including keystrokes, code edits, file interactions, and window focus changes. By analyzing this data, the framework offers deep insights into the quality and efficiency of students' coding practices. This process-based approach allows educators to understand not only **what** was submitted but **how** it was developed. These insights have shown strong correlations with final grades and provide a valuable foundation for real-time feedback, student support, and fairer assessments.

Unlike traditional grading systems that focus on outcomes alone, AssessMe places emphasis on **the process of learning**–capturing code-in-time patterns and coding behaviors that reflect student engagement, effort, and problem-solving strategies. This becomes particularly important in the context of generative AI tools, where code completion may not reflect genuine learning. AssessMe also supports academic integrity by helping to identify potential cases of automated or copied solutions, while simultaneously offering instructors predictive indicators of performance and areas where students struggle.

The main contribution of this article is the use of AssessMe's learning indicators to evaluate student coding behavior and predict academic outcomes. We demonstrate how indicators such as active coding time, code modifications, and code-in-time patterns–how students write code across the assignment duration–can serve as reliable predictors of engagement, learning progression, and final performance. These metrics enable early detection of students at risk, promote fair and evidence-based evaluation, and offer a novel approach to process-driven assessment in programming education.

This article is organized as follows. Section II presents the AssessMe framework and outlines the data collection process along with the construction of learning indicators. Section III begins with a statistical analysis of the collected data and proceeds to the development, training, and validation of predictive models. Finally, Section V summarizes the key findings and discusses directions for future work.

II. METHODOLOGY

The primary objective of this research is to utilize Educational Data Mining techniques in Computer Science courses to systematically collect and analyze targeted data for the accurate prediction of student performance, particularly in terms of pass/fail outcomes, during the early stages of the course. The initial and most crucial phase involves data selection and acquisition, guided by the following requirements for the data acquisition framework:

- Seamless integration into the existing programming course structure, which includes different types of assignments and LMS platforms, and
- focused data collection relevant only to course assignment content, thereby minimizing privacy concerns.

A. AssessME

To achieve these objectives, we utilize the **AssessMe** framework [23], [24], which monitors and validates the assignment creation process in real time, generating rich learning indicators that enable effective and targeted educational data mining. By capturing the complete coding workflow, AssessMe supports high-quality teaching and learning while reinforcing academic integrity within educational institutions.

The AssessMe framework, illustrated in Figure 1, integrates an AssessMe extension into the preferred Integrated Development Environment (IDE) used in the course. This extension is designed for data acquisition. Currently, the AssessMe extension is implemented exclusively for Visual Studio Code (VS Code), which was chosen due to its widespread use in programming education and its robust extension API. The primary function of the AssessMe Extension is to record all activities related to assignment creation. This includes capturing snapshots of source code changes, along with other data such as keystrokes and the number of times the IDE window is minimized or maximized. All captured data are consolidated into a single file, known as the AssessMe File, which is automatically generated alongside the source file. Upon completion of the assignment, students must submit both the source code and the AssessMe file, which validates the process of creating the source code, to their graders.

While the current implementation is limited to VS Code, we emphasize that this restriction applies only to the data collection component. The rest of the framework–including

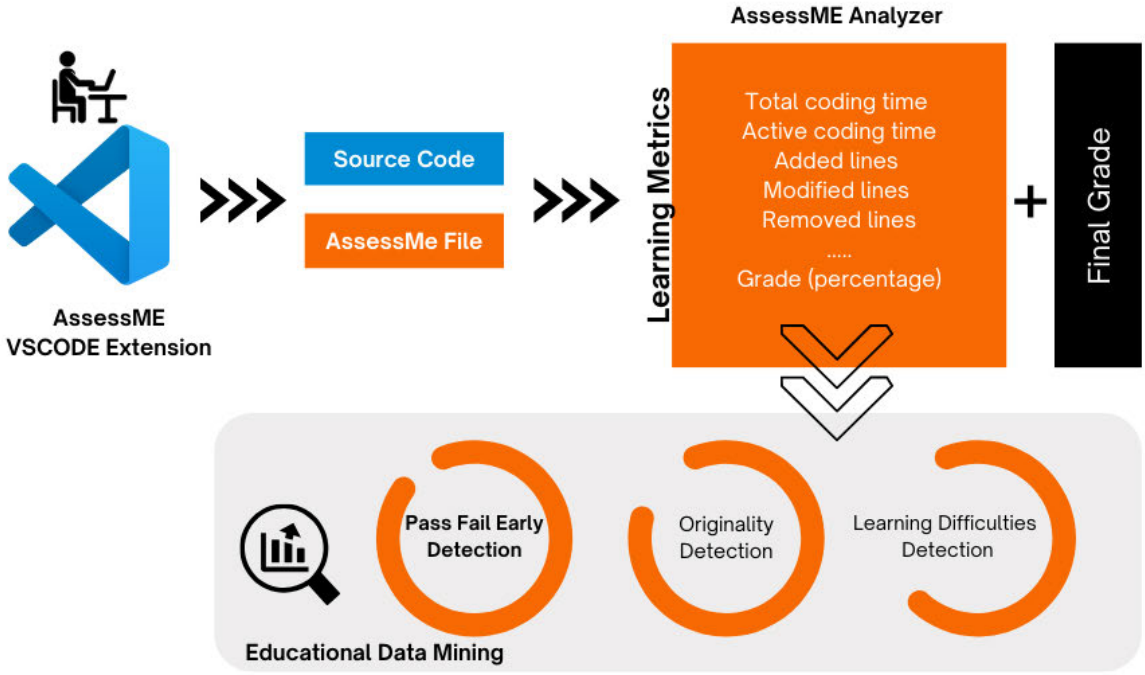


FIGURE 1. AssessMe EDM Framework.

the AssessMe Analyzer, machine learning models, and reporting modules–is entirely platform-agnostic. Therefore, extending AssessMe to other IDEs (e.g., PyCharm, Eclipse, or browser-based editors) would involve only developing similar lightweight data collection extensions. We plan to pursue this in future iterations to support broader adoption across diverse teaching environments.

For every source code, the AssessMe Analyzer generates learning indicators, such as total and active coding time, as well as added, modified, or removed lines, among others. These indicators are used in educational data mining to build models that can predict the originality of the work, identify learning difficulties, or enable early detection of pass/fail outcomes, which is the focus of this article.

An earlier exploratory classroom deployment of AssessMe [24] helped guide the current design and methodology. However, that initial study encountered major data consistency challenges–several weeks of student activity data were lost due to ongoing changes in software configuration during the semester. As a result, the findings from that effort were not included in this article. Nonetheless, it provided valuable insights that led to improvements in the framework's stability, data pipeline robustness, and deployment strategy, all of which shaped the work presented here.

B. COLLECTED DATA

The Software Development & Problem Solving I and II courses, also known as Programming I and II, are essential for first-year Information Technology students at the Rochester Institute of Technology in Croatia. These courses must be completed to enroll in advanced programming courses. Failure to pass these foundational courses can prevent students from advancing to the next academic year, potentially delaying graduation by at least a year. Moreover, these courses play a vital role in student retention, significantly influencing the number of students who persist in the program.

Both courses consist of multiple types of assignments, which are shortly described below, as they play important role in the pass/fail prediction:

- **Homework assignments (HW):** Students complete these assignments independently outside of the classroom without any supervision. Assistance from teaching assistants and faculty members is available if needed. Each assignment typically has a deadline of one week, with a total of 15-20 assignments during the course.
- **Practicals (P):** These are fully supervised exam assignments. There are 4 practicals scheduled throughout the semester, each with a time limit of 2 hours.
- **Mini Practicals (MP):** These are shorter, fully supervised exam assignments. The course includes about ten mini practicals, each with a time constraint of 30 minutes.
- **Class Activity (CA):** During lectures, the faculty presents multiple small coding assignments that students are encouraged to complete in real-time. Participation in these activities earns students small bonus points.
- **Problem Solving:** This involves various group projects that are primarily focused on non-coding tasks.

To pass the course, students must achieve more than 50% on the Practical assignments and a minimum overall score of 60% across all assignments to earn a grade of "D".

During the academic year 2024/2025, 50 students were enrolled in Programming 1 and 36 in Programming 2, as shown in Table 1. Altogether, these 86 students submitted 3,537 assignments, each accompanied by its corresponding AssessMe file. As shown in Table 2, a total of 70 students (81.4%) successfully passed the courses, while 16 students (18.6%) either failed or withdrew.

TABLE 1. Total number of students and assignments in Programming 1 and 2 for the academic year 2024/2025.

	N	HW	P	MP
Programming 1	50	689	455	404
Programming 2	36	824	648	517
Total	86	1513	1103	921

TABLE 2. Pass/fail distribution of students in Programming 1 and 2 during the academic year 2024/2025.

	Passed	F/W	Total	% F/W
Programming 1	40	10	50	20.0%
Programming 2	30	6	36	16.7%
Total	**70**	**16**	**86**	**18.6%**

C. LEARNING INDICATORS

All assessments were required to be completed using AssessMe. Submissions without an AssessMe file received a score of zero. The AssessMe Analyzer subsequently extracted learning behavior indicators, which served as features for training predictive models aimed at classifying pass/fail outcomes. These indicators include:

- **Total coding time (TCT)** or total time (in minutes) that the IDE was opened and focused. TCT reflects the total investment in time and the potential opportunity for learning.
- **Active coding time (ACT)** or the total time in minutes spent actively coding in the IDE. ACT captures focused effort and is a stronger signal of engagement than TCT alone.
- **Added lines (AL)** or the total number of newly added code lines.High AL suggests experimentation or initial implementation work.
- **Modified lines (ML)** or number of edited lines, which includes changes made to previously written code.
- **Removed lines (RL)** or the total number of removed code lines. RL often correlates with code refinement and debugging activity.

These indicators, TCT, ACT, AL, ML, RL, and their derivatives are frequently used in real-world software development environments, such as version control systems (e.g., Git), code quality analytics, and developer performance assessments. Their inclusion here enables a grounded and interpretable analysis of programming behavior in an educational context.

In addition to these behavioral indicators, grade-based data are used:

- **Grades (G)** – the numeric assessment for each individual assignment.
- **Final grade (FG)** – the cumulative score reflecting overall performance throughout the course.

III. RESULTS

To develop a pass/fail retention model, 1,538 assignments from Programming 1 and 1,989 assignments from Programming II were analyzed. The initial stage of data mining involves statistical analysis to validate any initial correlations with the intended outcome. Before proceeding with the analysis, it is important to note the following:

- Students were introduced to the AssessMe system, and learning indicators were employed to detect any potential unethical behaviors, such as cheating. The findings discussed in this research were not disclosed to the students during the course.
- Programming II is a continuation of Programming I, involving the students who passed Programming I.

A. LEARNING INDICATORS STATISTICAL DATA ANALYSIS

Statistical analysis was performed separately for each course and type of assignment, differentiated by varying levels of supervision and time constraints. For each assignment, the average learning indicators were calculated and are presented as a percentage of the mean value. Furthermore, assignments were categorized according to the final grade (US Standard Grades [25]), with the results displayed in Tables 5 through 3. The first column, titled FG shows the final grade earned in the course, while the last column, G indicates the average grade across all collected assignments.

The statistical analysis of all assignments in Programming I is presented in Table 5. The data reveals that students receiving final grades of "D" and "F" spent, on average, only 84% and 82% of the mean active coding time, respectively. In contrast, students with higher grades consistently exceeded the average, with their active coding times ranging from 102% to 105%. The same observation can be applied to all learning indicators. Further analysis reveals that students who received a grade of "D" had average percentages of 68% for modified lines and 50% for removed lines. A lower percentage of removed lines in unsupervised assignments often indicates that these assignments may have been copied from another source, resulting in fewer necessary modifications and corrections.

Table 4 presents the statistical analysis of the homework assignments. These assignments, completed independently outside the classroom, require the most time from students. They play a crucial role in enhancing programming skills and are essential for preparing students for the Practical assignments. Notably, students with final grades of "D" and "F" achieved a high "C" grade on these assignments, yet the learning indicators for "F" students were all below 76%. This suggests that these students did

TABLE 3. Mean percentage values of learning indicators across all assignments in Programming I.

FG	TCT %	ACT %	AL %	ML %	RL %	G
A	101	102	103	112	94	A
B	104	104	105	104	109	B
C	103	105	103	101	118	C
D	105	102	106	107	78	C
F	88	**84**	71	**68**	**50**	D
W	81	**82**	89	80	87	D

TABLE 4. Mean percentage values of learning indicators across homework assignments in Programming I.

FG	TCT %	ACT %	AL %	ML %	RL %	G
A	118	117	110	124	110	A
B	101	100	100	99	96	A
C	105	106	109	101	123	B
D	87	87	83	102	80	**C**
F	**75**	**76**	**75**	**67**	**57**	**C**
W	**74**	**75**	**83**	**68**	**72**	D

not spend sufficient time coding independently, leading to inadequate skill development and poor performance on the Practical assignments. These findings also highlight the risk of relying solely on grades for pass/fail predictions, as grades may not accurately reflect a student's true knowledge.

Table 5 presents the statistical analysis of the Programming II homework assignments. Unlike in Programming I, students who received "F" and "W" grades in this course demonstrated above-average activity in nearly all learning indicators. This likely reflects their prior experience from Programming I and their decision to invest more time in independent coding. However, despite the increased effort on individual assignments, the key difference lies in the overall number of submissions.

TABLE 5. Mean percentage values of learning indicators across homework assignments in Programming II.

FG	TCT %	ACT %	AL %	ML %	RL %	G
A	113	117	104	122	103	A
B	94	95	108	109	118	A
C	93	91	91	91	89	C
D	96	97	81	83	79	B
F	102	103	108	86	142	C
W	**149**	**138**	**179**	**124**	**130**	D

Finally, Table 6 displays the average number of submissions per student, organized by grade. This table highlights the reason for the failure of students in Programming II, as their number of submissions is significantly lower compared to those who passed.

B. CUMULATIVE LEARNING INDICATORS

Building on the statistical analysis, accurately predicting pass/fail outcomes requires monitoring both the learning indicators for each individual assignment and the total number of submissions. By combining these two indicators into a cumulative learning indicators (CLI), we gain a more complete view of a student's progress. This CLIs offers a clearer understanding of the quality of work on each assignment and the student's ongoing engagement throughout the course, enabling more precise predictions and better-targeted interventions.

TABLE 6. Average number of submissions per student organized by grade in Programming I and II.

Final Grade	Submissions P1	Submissions P2
A	17	27
B	16	27
C	12	22
D	14	27
F	10	7
W	7	9

Figure 2 illustrates the CLI for ActiveCodingTime, AddedLines, ModifiedLines, and RemovedLines in the Programming I course. The activity spans from September 15th to December 15th, with the data resampled on a weekly basis. Each line series represents the activity of an individual student: gray lines indicate passing grades, orange lines correspond to "F" grades, and red lines represent "W" grades. The blue dashed lines show the average values. The results indicate that failing students are generally positioned below the average line across all indicators. These time series data will form the basis for developing a predictive model designed to identify students at risk of failing. By analyzing trends and patterns within the CLI, the model will be capable of forecasting pass/fail outcomes as early as the fifth week of the course.

C. PREDICTIVE MODEL FRAMEWORK

Cumulative learning indicators, represented as time series data, provide insights into a student's behavior over time. To effectively apply supervised and unsupervised machine learning algorithms, it is crucial to extract meaningful features from these time series, such as minimum, maximum, and mean values. These features are then utilized by machine learning algorithms to identify correlations with the target variable.

1) TIME SERIES FEATURES

Although most supervised machine learning algorithms are not inherently designed for sequential or temporal data, time series can still be effectively leveraged by transforming them into a feature-based representation. This transformation involves extracting statistical, structural, and frequency-domain features from the raw sequences, thereby converting time-dependent patterns into a format suitable for conventional algorithms like decision trees, random forests, support vector machines, and gradient boosting models [26], [27]. By capturing the temporal dynamics through these derived features, the models can indirectly

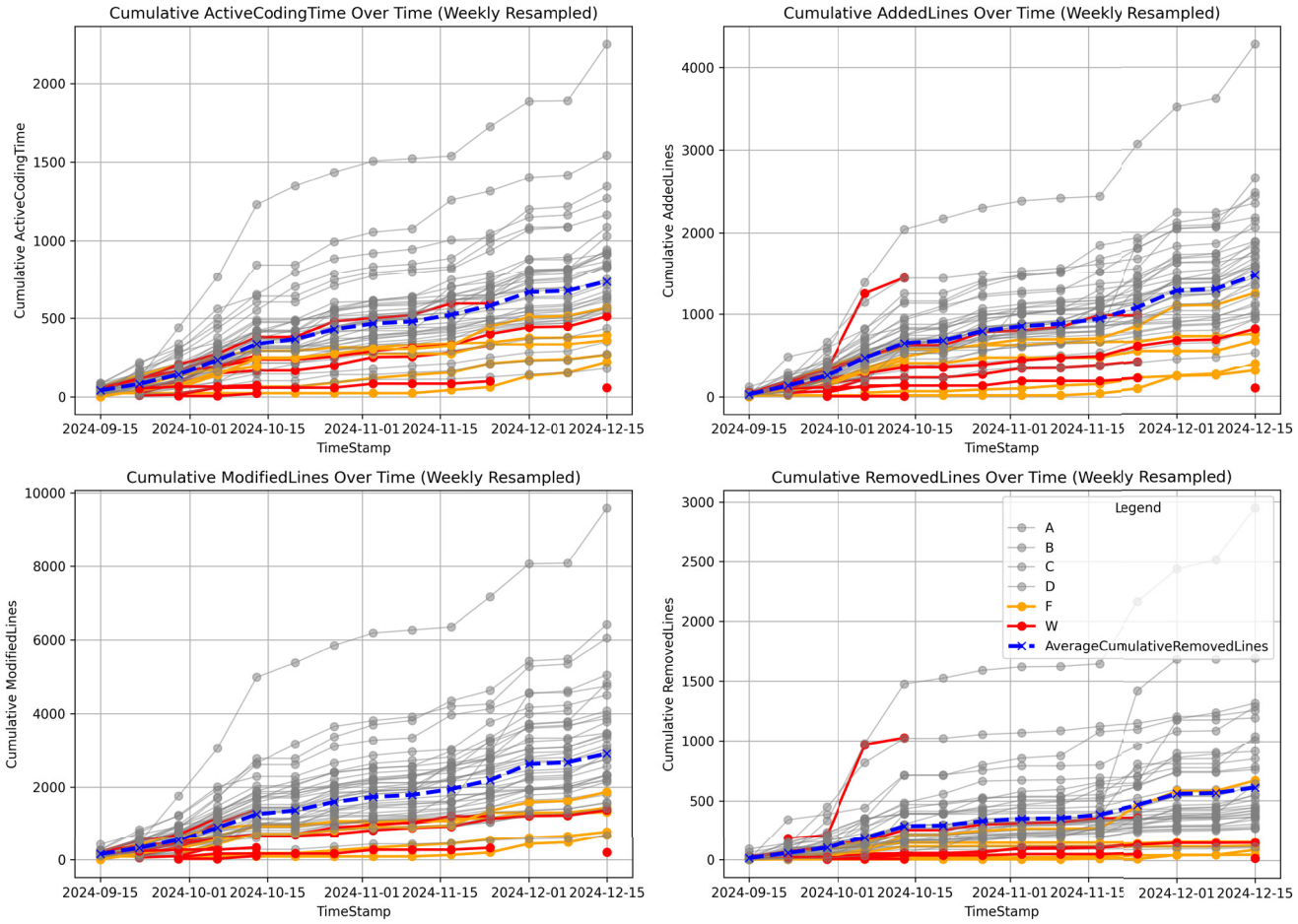
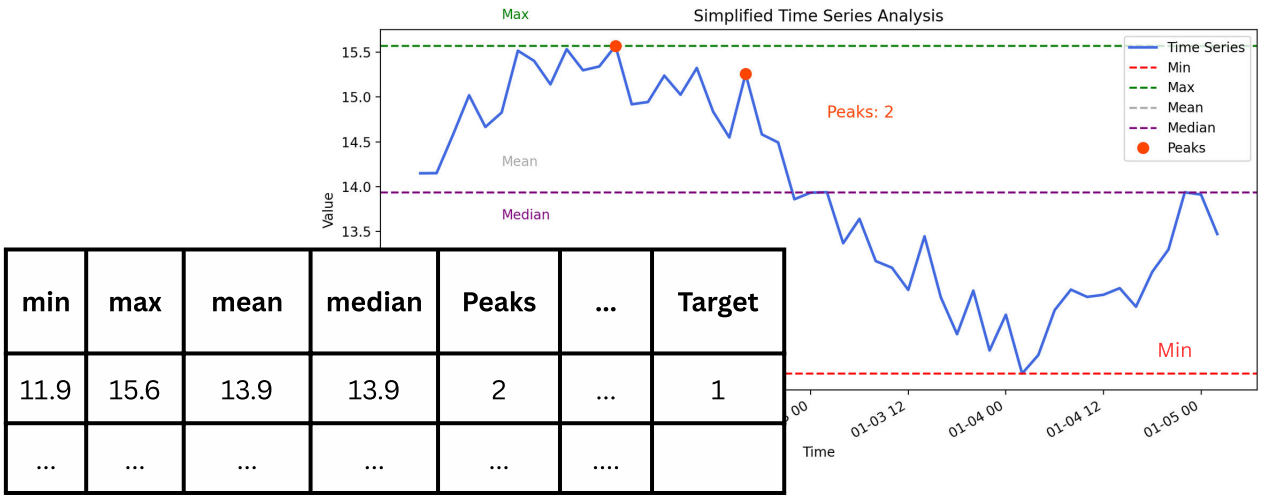


FIGURE 2. Cumulative learning indicators over time in the Programming I course. Gray lines represent passing grades, orange lines denote "F" grades, and red lines indicate "W" grades. The blue dashed lines depict the average values.



min	max	mean	median	Peaks	...	Target
11.9	15.6	13.9	13.9	2	...	1
...	...	...	...	...	....	

FIGURE 3. Feature extraction from time series using TSFRESH for supervised learning.

learn from the underlying sequence of events without requiring specialized time series algorithms such as RNNs or LSTMs. This approach allows us to harness the predictive power of general-purpose machine learning models while still preserving the temporal context of student learning behavior.

Manually extracting relevant features from time series data is often time-consuming and complex. To streamline this process, we utilize the Python package TSFRESH (Time Series FeatuRe Extraction on basis of Scalable Hypothesis tests) [27], which automates feature extraction using 63 time series characterization methods to compute 794 features by default. To enhance relevance, TSFRESH employs the FRESH algorithm to filter out non-informative features, retaining only those that show a statistically significant correlation with the target variable.

Figure 3 provides a simplified example of how the observerd time series data is transformed into a structured format suitable for supervised learning. Key statistical features–such as minimum, maximum, mean, median, and the number of peaks–are extracted and organized into a feature table, where each row represents a single time series instance and the final column contains the corresponding target variable. This entire process is automated by TSFRESH, which not only computes a comprehensive set of features but also selects the most predictive ones through hypothesis testing, resulting in a more focused and effective learning dataset.

2) SUPERVISED MACHINE LEARNING

To predict student outcomes (PASS/FAIL), we implemented three supervised algorithms: Decision Tree (for interpretability), XGBoost (for strong predictive performance and robustness), and k-Nearest Neighbors (k-NN) (as an intuitive baseline) [28]. Due to pronounced class imbalance in the dataset (i.e., a higher prevalence of PASS instances over FAIL), we applied Synthetic Minority Over-sampling Technique (SMOTE) [28] during training to mitigate bias toward the majority class. Model validation employed Stratified 10-Fold Cross-Validation, ensuring consistent class distribution across folds–a critical step for robust evaluation in imbalanced settings.

The failure/withdrawal rate in our dataset is 18.6%, which falls well within the recommended SMOTE range of 10–40% for effective oversampling [29], [30], [31]. Specifically, the failure/withdrawal rate in Programming I is 20.0% (10 out of 50 students), and in Programming II it is 16.7% (6 out of 36 students)–both comfortably within the target range. This supports the methodological appropriateness of applying SMOTE in our context, as this range ensures sufficient minority representation without introducing excessive synthetic variance.

To mitigate the risk of overfitting–particularly given the small number of unique students and the large number of extracted features–we applied TSFRESH's built-in relevance filtering to retain only statistically significant features related to the target variable. This reduced dimensionality and focused model training on the most informative indicators.

Moreover, overfitting is mitigated by the structure of our dataset: instead of artificially increasing the data through feature engineering, we rely on Normalized Course Learning Indicators (NCLIs)–weekly behavioral metrics such as active coding time, added lines, and removed lines. These indicators result in a rich, temporally distributed dataset with thousands of training instances, enabling robust and generalizable model learning despite the relatively small number of unique students.

In addition, we used regularized models (e.g., depth-limited trees in XGBoost) and avoided information leakage by ensuring that temporal dependencies were not violated during validation. This combination of statistical filtering, regularization, and time-aware design helps ensure that the models generalize beyond the observed data.

Model performance was evaluated using accuracy, F1 score, and confusion matrix components (true positives [TP], false positives [FP], true negatives [TN], and false negatives [FN]). While accuracy offers a general measure of correctness, it can be misleading in imbalanced datasets, as it may obscure poor performance in detecting minority-class instances (e.g., FAIL predictions). To address this, the F1 score was prioritized, as it balances precision and recall by penalizing both false positives and false negatives [32].

D. BUILDING A TRAINING DATASET

To build a training dataset, the following steps were undertaken:

1) **Calculation of Learning Indicators**: Determine the learning indicator for each student's assessment.
2) **Weekly Cumulative Learning Indicator**: Calculate cumulative learning indicator, resampling the data on a weekly basis.
3) **Normalized Cumulative Learning Indicator (NCLI)**: Normalize the cumulative learning indicator around the average weekly values.
4) **Time Series Extraction**: Extract time series data from the NCLIs for multiple observation intervals ranging from D_4 to D_{15}. Here, D_4 represents data collected up to Week 4, and D_{15} encompasses all data collected to date. This approach allows the model to reliably generate predictions starting from Week 4 onwards.
5) **Feature Calculation Using TSFRESH**: Calculate TSFRESH features for all extracted time series across the observed times and automatically extract the most relevant features.
6) **Target Value Assignment**: For each set of features calculated in the previous step, assign a final grade as the target value.

It is crucial to emphasize the significance of constructing training sets for various observed times. Training a model solely on D_{15} has demonstrated poor performance in predicting the final grade using only earlier week data, such as data from Week 4 or Week 5. This discrepancy arises because different TSFRESH features are extracted when training on D_{15} as opposed to using multiple intervals from D_4 to D_{15}. For instance, the feature 'maximal value' may be relevant in D_{15}, but loses relevance when training across the broader data range from D_4 to D_{15}.

TABLE 7. Average classification report for used methods in Programming I.

Methods		Precision	Recall	F1	Support
Random Forest	PASS	0.97	0.97	0.97	456
	FAIL	0.94	0.93	0.94	192
Decision Tree	PASS	0.96	0.93	0.95	456
	FAIL	0.86	0.92	0.89	192
k-NN	PASS	0.95	0.89	0.91	456
	FAIL	0.77	0.88	0.82	192
XGBoost	PASS	0.97	0.97	0.97	456
	FAIL	0.93	0.92	0.92	192

E. TRAINING AND VALIDATION A PREDICTIVE MODEL FOR PASS/FAIL OUTCOMES

The proposed framework for constructing the predictive model is depicted in Figure 4. It is essential that the features defined during the training phase are consistently used when analyzing new, unseen data.

1) PREDICTIVE MODEL FOR PROGRAMMING I

The dataset comprises 4,796 NCLIs spanning from D_4 to D_{15}. TSFRESH extracted 546 features for each learning indicator, amounting to a total of 2,256 features that correlate with the target value. Utilizing 5-fold cross-validation, the average classification report for various methods is presented in Table 7. The results indicate a strong correlation between the features and the target value, leading to highly accurate predictions. The XGBoost method achieved the highest F_1 score of 0.97 for "PASS" and 0.93 for "FAIL" predictions.

The Figure 5 illustrates the average F_1 score across various observed time intervals D_2 to D_{10}. The initial observation is D_2 (2024-09-17), marking the end of the second week of the semester. The model's performance demonstrates significant improvement through D_4 (2024-10-01) and maintains excellent results after D_6 (2024-10-15). The results indicate that while the model shows some uncertainty for certain students between D_4 and D_6, it achieves 100% prediction accuracy beyond this period. By the end of Week 5, the model demonstrates an average F_1 score of 97% for "PASS" and 93% for "FAIL."

2) PREDICTIVE MODEL FOR PROGRAMMING II

The dataset encompasses 4,691 NCLIs covering the range from D_4 to D_{15}. Using TSFRESH, 549 features were extracted for each learning indicator, totaling 2,198 features associated with the target value. We implemented 5-fold cross-validation, and the average classification report for various methods is detailed in Table 8.

Once again, the results reveal a pronounced correlation between the features and the target value, leading to predictions that are even more accurate than those in Programming I. The XGBoost method reached the highest F_1 scores of 0.99 for "PASS" and 0.98 for "FAIL" predictions.

TABLE 8. Average classification report for used methods in Programming II.

Methods		Precision	Recall	F1	Support
Random Forest	PASS	0.98	0.99	0.98	390
	FAIL	0.98	0.94	0.95	104
Decision Tree	PASS	0.96	0.93	0.95	390
	FAIL	0.98	0.99	0.98	104
k-NN	PASS	0.97	0.92	0.94	390
	FAIL	0.75	0.90	0.82	104
XGBoost	PASS	0.99	0.99	0.99	390
	FAIL	0.99	0.98	0.98	104

TABLE 9. Average classification report for used methods in combined Programming I and II.

Methods		Precision	Recall	F1	Support
Random Forest	PASS	0.98	0.98	0.98	846
	FAIL	0.96	0.95	0.95	296
Decision Tree	PASS	0.97	0.95	0.96	846
	FAIL	0.87	0.92	0.89	296
k-NN	PASS	0.89	0.79	0.83	846
	FAIL	0.55	0.71	0.62	296
XGBoost	PASS	0.98	0.99	0.99	846
	FAIL	0.97	0.95	0.96	296

The Figure 6 illustrates the F_1 score across various observed time intervals D_2 to D_{10}. The initial observation is D_2 (2025-01-28). By the end of Week 5, the model demonstrates an average F_1 score of 98% for "PASS" and 93% for "FAIL."

3) COMBINED PREDICTIVE MODEL FOR PROGRAMMING I AND II

Given the similar structure and workload of the courses, the dataset from both can be utilized to construct a combined predictive model. The average classification report for various methods is displayed in Table 9.

The Figure 7 shows the F_1 results for the combined model in Programming I. While the F_1 score at D_5 remains consistent with the non-combined model, a notable improvement is observed at D_6, where the combined model achieves a high F_1 score of score of 100% for both pass and fail predictions. Figure 8 shows the F_1 results for the combined model in Programming II. The combined model achieves a high score of 100% starting from the end of Week 5, outperforming the initial model. A detailed numerical comparison of these results is further illustrated in Table 14 and 15 in Appendix.

IV. LIMITATIONS

This study is based on a modest sample of 86 students from two programming courses, supported by 9,487 weekly engagement records (NCLIs) collected over one academic year. Although the sample size is limited, each student contributed multiple time-series engagement indicators, increasing the effective dataset size. We acknowledge the potential risk of overfitting–particularly due to the high dimensionality of features extracted via TSFRESH relative to the number of unique students. To address this, we employed TSFRESH's built-in statistical filtering to retain only relevant

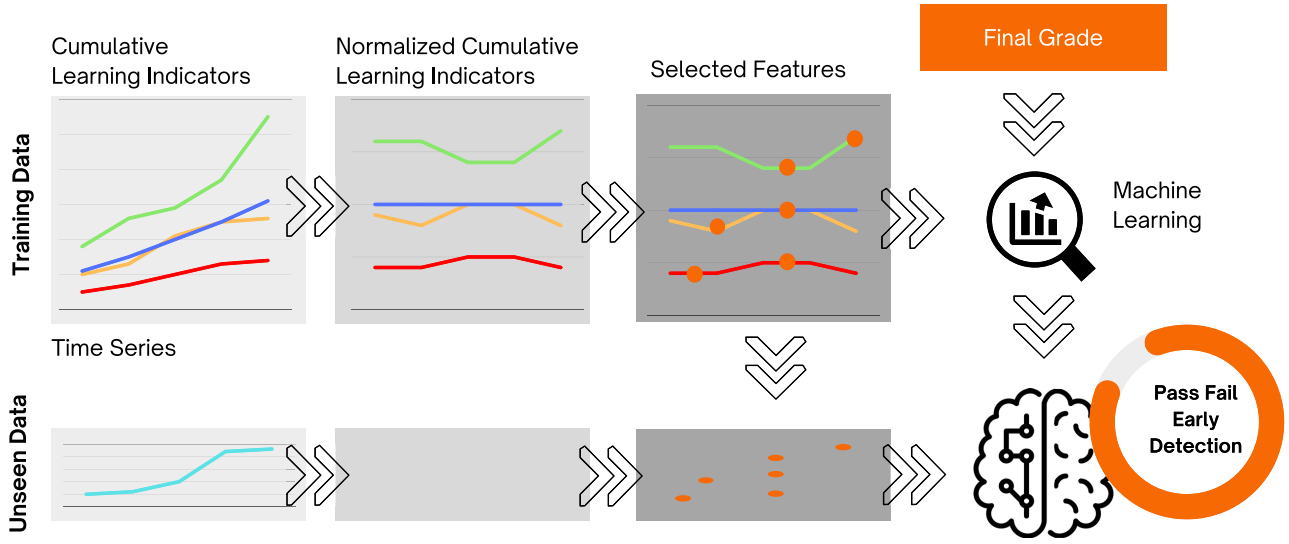


FIGURE 4. The proposed framework for building the predictive model based on cumulative learning indicators.

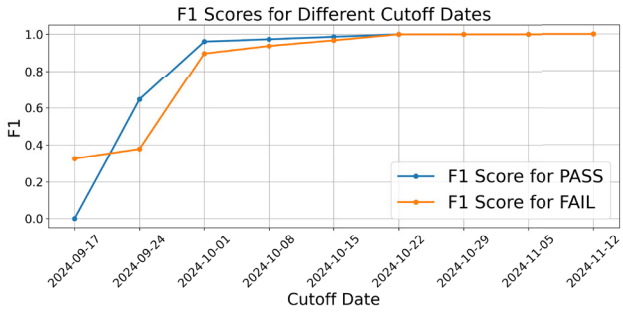


FIGURE 5. Average F1 score across cutoff dates for Programming I; full numerical results are provided in Table 10 in Appendix.

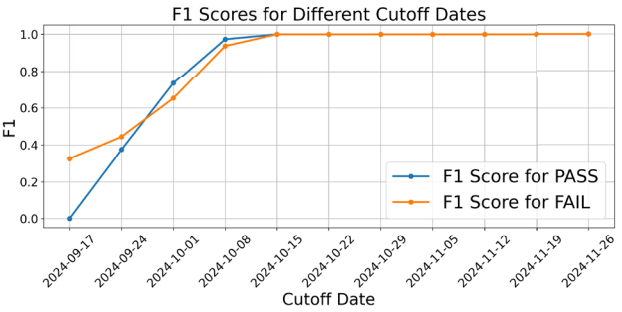


FIGURE 7. Combined model analysis: average F_1 scores across different cutoff dates in Programming I; full numerical results are provided in Table 12 in Appendix.

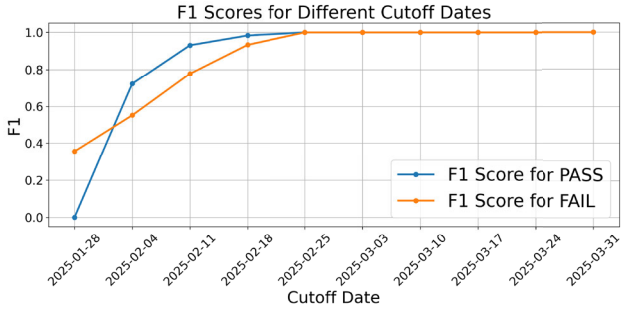


FIGURE 6. Average F1 Scores Across Different Cutoff Dates in Programming II; full numerical results are provided in Table 11 in Appendix.

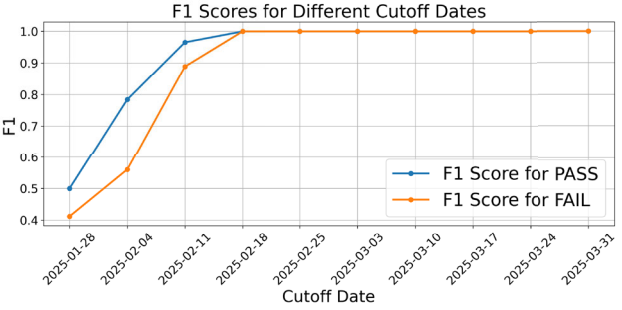


FIGURE 8. Combined model dnalysis: average F_1 scores across different cutoff dates in Programming II; full numerical results are provided in Table 13 in Appendix.

features, used regularized models with temporal validation to prevent leakage, and relied on interpretable engagement-based features (e.g., active coding time, lines modified) grounded in educational theory. We also recognize that using SMOTE on small datasets can introduce synthetic bias. However, our application falls within recommended class imbalance ratios, and no artificial feature expansion was performed.

While cross-institutional validation remains a necessary next step, the behavioral patterns observed are consistent with

TABLE 10. Average F1 scores scross different cutoff dates in Programming I.

Week	Cutoff Date	F1 Pass	F1 Fail
D_2	2024-09-17	0.000000	0.326531
D_3	2024-09-24	0.646154	0.378378
D_4	2024-10-01	0.961039	0.896552
D_5	2024-10-08	0.973684	0.937500
D_6	2024-10-15	0.987013	0.967742
D_7	2024-10-22	1.000000	1.000000
D_8	2024-10-29	1.000000	1.000000
D_9	2024-11-05	1.000000	1.000000
D_{10}	2024-11-12	1.000000	1.000000

TABLE 11. Average F1 scores across different cutoff dates in Programming II.

Week	Cutoff Date	F1 Pass	F1 Fail
D_2	2025-01-28	0.000000	0.355556
D_3	2025-02-04	0.723404	0.551724
D_4	2025-02-11	0.931034	0.777778
D_5	2025-02-18	0.983607	0.933333
D_6	2025-02-25	1.000000	1.000000
D_7	2025-03-03	1.000000	1.000000
D_8	2025-03-10	1.000000	1.000000
D_9	2025-03-17	1.000000	1.000000
D_{10}	2025-03-24	1.000000	1.000000

established educational literature. The framework's broader relevance is supported by international recognition, including the 2024/25 PROFFORMANCE Award [33]. Future work will focus on external replication and scaling across diverse institutional contexts.

V. CONCLUSION

This study demonstrates the potential of the AssessMe framework to improve retention and performance in introductory programming courses by providing actionable insights into student learning behaviors. The results show that learning indicator, when analyzed using advanced predictive models, can accurately forecast pass/fail outcomes as early as the fifth week, enabling timely and targeted interventions. With models achieving over 93% F_1 score, this approach underscores the value of integrating real-time educational data mining into teaching practices to address the high dropout rates prevalent in programming education.

While the findings are promising, they are based on two courses–Programming I and II–within the Web & Mobile Computing program at RIT Croatia. These courses, although foundational, represent a specific structure and context. Future research should explore the application of this model across courses with varying structures, assessment types, and disciplines to validate its generalizability. Additionally, its use with upcoming generations of students will offer deeper insights into longitudinal trends and the evolving dynamics of programming education.

Building on this dataset, our future work will delve into originality detection, where we aim to evaluate the authenticity of student assignments using learning indicators tailored to specific tasks. This includes distinguishing between original student work and code potentially generated by tools such as ChatGPT or reused from peers. Preliminary findings suggest that such externally assisted submissions often exhibit notably reduced active coding time and lower engagement–patterns detectable through AssessMe. However, a more detailed empirical analysis is needed to differentiate between various sources of assistance.

TABLE 12. Combined model analysis: average F1 scores across different cutoff dates in Programming I.

Week	Cutoff Date	F1 Pass	F1 Fail
D_2	2024-09-17	0.000000	0.326531
D_3	2024-09-24	0.375000	0.444444
D_4	2024-10-01	0.733333	0.652174
D_5	2024-10-08	0.973684	0.937500
D_6	2024-10-15	1.000000	1.000000
D_7	2024-10-22	1.000000	1.000000
D_8	2024-10-29	1.000000	1.000000
D_9	2024-11-05	1.000000	1.000000
D_{10}	2024-11-12	1.000000	1.000000

TABLE 13. Combined Model Analysis: Average F_1 scores across different cutoff dates in Programming II.

Week	Cutoff Date	F1 Pass	F1 Fail
D_2	2025-01-28	0.500000	0.411765
D_3	2025-02-04	0.784314	0.560000
D_4	2025-02-11	0.965517	0.888889
D_5	2025-02-18	1.000000	1.000000
D_6	2025-02-25	1.000000	1.000000
D_7	2025-03-03	1.000000	1.000000
D_8	2025-03-10	1.000000	1.000000
D_9	2025-03-17	1.000000	1.000000
D_{10}	2025-03-24	1.000000	1.000000

TABLE 14. Comparison of average F1 scores across different cutoff dates in Programming I. The *Model 1* represents the initial predictive model.

Week	Cutoff Date	Model 1		Combined	
		F1 Pass	F1 Fail	F1 Pass	F1 Fail
D_2	2024-09-17	0.00	0.33	0.00	0.33
D_3	2024-09-24	0.65	0.38	0.38	0.44
D_4	2024-10-01	0.96	0.90	0.73	0.65
D_5	**2024-10-08**	**0.97**	**0.94**	**0.97**	**0.94**
D_6	**2024-10-15**	**0.99**	**0.97**	**1.00**	**1.00**
D_7	2024-10-22	1.00	1.00	1.00	1.00
D_8	2024-10-29	1.00	1.00	1.00	1.00
D_9	2024-11-05	1.00	1.00	1.00	1.00
D_{10}	2024-11-12	1.00	1.00	1.00	1.00

TABLE 15. Comparison of average F1 scores across different cutoff dates in Programming II. The *Model 1* represents the initial predictive model.

Week	Cutoff Date	Model 1		Combined	
		F1 Pass	F1 Fail	F1 Pass	F1 Fail
D_2	2025-01-28	0.00	0.36	0.50	0.41
D_3	2025-02-04	0.72	0.55	0.78	0.56
D_4	**2025-02-11**	**0.93**	**0.78**	**0.97**	**0.89**
D_5	**2025-02-18**	**0.98**	**0.93**	**1.00**	**1.00**
D_6	2025-02-25	1.00	1.00	1.00	1.00
D_7	2025-03-03	1.00	1.00	1.00	1.00
D_8	2025-03-10	1.00	1.00	1.00	1.00
D_9	2025-03-17	1.00	1.00	1.00	1.00
D_{10}	2025-03-24	1.00	1.00	1.00	1.00

Furthermore, by analyzing typing patterns captured through AssessMe, we plan to investigate stress pattern analysis. This research could contribute significantly to identifying learning difficulties, providing an additional layer of understanding and support to enhance student outcomes.

Finally, although AssessMe currently integrates with VS Code, its core analysis components are platform-independent. Future development will focus on extending compatibility to additional coding environments, supporting broader institutional adoption and ensuring long-term scalability.

APPENDIX
SUPPLEMENTARY PERFORMANCE TABLES

This appendix provides supplementary performance tables presenting the complete numerical results that support Figures 5, 6, 7, and 8.

ACKNOWLEDGMENT

The authors extend their gratitude to Rochester Institute of Technology for its support through the Provost's Learning Innovation Grants, which were instrumental in the development and implementation of this study. They also acknowledge the invaluable feedback and technical assistance provided by the AssessMe Team, whose expertise and dedication greatly enriched the quality and scope of this work.

REFERENCES

[1] B. Gutiérrez-De-Rozas, E. C. Molina, and E. López-Martín, "Academic failure and dropout: Untangling two realities," *Eur. J. Educ. Res.*, vol. 11, pp. 2275–2289, Oct. 2022.

[2] S. Ram, Y. Wang, F. Currim, and S. Currim, "Using big data for predicting freshmen retention," in *Proc. Int. Conf. Interact. Sci.*, 2015.

[3] V. Tinto, "Enhancing student success: Taking the classroom success seriously," *Int. J. 1st Year Higher Educ.*, vol. 3, no. 1, pp. 1–8, Mar. 2012.

[4] J. M. Braxton, A. S. Hirschy, and S. A. McClendon, *Understanding and Reducing College Student Departure*. Hoboken, NJ, USA: Wiley, 2004.

[5] Eurostat Statistics. *Early Leavers From Education and Training*. European Commission, Luxembourg. [Online]. Available: https://ec.europa.eu/eurostat/statistics-explained/index.php

[6] J. M. Ortiz-Lozano, A. Rua-Vieites, P. Bilbao-Calabuig, and M. Casadesús-Fa, "University student retention: Best time and data to identify undergraduate students at risk of dropout," *Innov. Educ. Teaching Int.*, vol. 57, no. 1, pp. 74–85, Aug. 2018.

[7] L. E. Margulieux, B. B. Morrison, and A. Decker, "Reducing withdrawal and failure rates in introductory programming with subgoal labeled worked examples," *Int. J. STEM Educ.*, vol. 7, no. 1, May 2020.

[8] J. Bennedsen and M. E. Caspersen, "Failure rates in introductory programming: 12 years later," *ACM Inroads*, vol. 10, no. 2, pp. 30–36, Apr. 2019.

[9] G. D. Caruth, "Student engagement, retention, and motivation: Assessing academic success in today's college students," *Participatory Educ. Res.*, vol. 5, no. 1, pp. 17–30, Dec. 2018.

[10] B. Albreiki, N. Zaki, and H. Alashwal, "A systematic literature review of Student' performance prediction using machine learning techniques," *Educ. Sci.*, vol. 11, no. 9, p. 552, Sep. 2021.

[11] Y. B. Ampadu, "Handling big data in education: A review of educational data mining techniques for specific educational problems," *AI, Comput. Sci. Robot. Technol.*, vol. 2, Apr. 2023.

[12] D. Khairy, N. Alharbi, M. A. Amasha, M. F. Areed, S. Alkhalaf, and R. A. Abougalala, "Prediction of student exam performance using data mining classification algorithms," *Educ. Inf. Technol.*, vol. 29, no. 16, pp. 21621–21645, May 2024.

[13] D. Ying and J. Ma, "Student performance prediction with regression approach and data generation," *Appl. Sci.*, vol. 14, no. 3, p. 1148, Jan. 2024.

[14] E. A. Saire-Peralta and M. D. C. Córdova-Martínez, "Predicting academic performance path using classification algorithms," *Int. J. Inf. Educ. Technol.*, vol. 13, no. 12, pp. 1890–1898, 2023.

[15] C. Romero and S. Ventura, "Educational data mining and learning analytics: An updated survey," *WIREs Data Mining Knowl. Discovery*, vol. 10, no. 3, p. e135, May 2020.

[16] A. Bralić and B. Divjak, "Integrating MOOCs in traditionally taught courses: Achieving learning outcomes with blended learning," *Int. J. Educ. Technol. Higher Educ.*, vol. 15, no. 1, p. 2, Dec. 2018.

[17] N. Hoić-Božić and M. H. Dlab, "Experiences with the integration of MOOCs into online courses through E-tivity," in *Proc. IEEE Learn. With MOOCS (LWMOOCS)*, Oct. 2023, pp. 1–5.

[18] D. Dwikoranto, W. A. Surasmi, B. Kurniawan, I. R. Dawana, and L. N. Bergsma, "Utilization of universities' massive online open courses in learning management system: Research trends and bibliometric analysis," *IJORER, Int. J. Recent Educ. Res.*, vol. 4, no. 3, pp. 383–398, May 2023.

[19] D. Zhidkikh, V. Heilala, C. Van Petegem, P. Dawyndt, M. Jarvinen, S. Viitanen, B. De Wever, B. Mesuere, V. Lappalainen, L. Kettunen, and R. Hämäläinen, "Reproducing predictive learning analytics in CS1: Toward generalizable and explainable models for enhancing student retention," *J. Learn. Anal.*, vol. 11, no. 1, p. 21, 2024.

[20] V. Isomöttönen, A.-J. Lakanen, and V. Lappalainen, "Less is more! Preliminary evaluation of multi-functional document-based online learning environment," in *Proc. IEEE Frontiers Educ. Conf. (FIE)*, Oct. 2019, pp. 1–5.

[21] F. J. M. Coelho, E. H. T. Oliveira, F. D. Pereira, D. B. F. Oliveira, L. S. G. Carvalho, E. J. P. Souto, M. Pessoa, R. Melo, M. A. P. D. Lima, and F. G. Nakamura, "Learning analytics in introductory programming courses: A showcase from the federal university of amazonas," *Revista Brasileira de Informática na Educação*, vol. 31, pp. 1089–1127, Dec. 2023.

[22] J. Llanos, V. A. Bucheli, and F. Restrepo-Calle, "Early prediction of student performance in CS1 programming courses," *PeerJ Comput. Sci.*, vol. 9, p. e1655, Oct. 2023.

[23] SmoothSoft Ltd. (2025). *Assessme: Learning Validation and Academic Integrity Framework*. Accessed: May 27, 2025. [Online]. Available: https://assessme.com.hr/

[24] A. Mutka, F. Zivkovic, M. Breka, and M. Zagar, "Student retention using advanced learning validation software tool and educational data mining: A case study," in *Proc. 17th Int. Technol., Educ. Develop. Conf. (INTED)*, vol. 1, Valencia, Spain, Mar. 2023, pp. 8492–8496.

[25] National Center for Education Statistics. (2015). *How is Grade Average Calculated*. U.S. Department of Education. Accessed: Feb. 14, 2015. [Online]. Available: https://nces.ed.gov/nationsreportcard/hsts/howgpa.aspx

[26] T. Henderson and B. D. Fulcher, "An empirical evaluation of time-series feature sets," in *Proc. Int. Conf. Data Mining Workshops (ICDMW)*, Dec. 2021, pp. 1032–1038.

[27] M. Christ, N. Braun, J. Neuffer, and A. W. Kempa-Liehr, "Time series FeatuRe extraction on basis of scalable hypothesis tests (tsfresh—A Python package)," *Neurocomputing*, vol. 307, pp. 72–77, Sep. 2018.

[28] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 3rd ed., Upper Saddle River, NJ, USA: Prentice-Hall, 2010.

[29] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *J. Artif. Intell. Res.*, vol. 16, pp. 321–357, Jun. 2002.

[30] A. Fernandez, S. Garcia, F. Herrera, and N. V. Chawla, "SMOTE for learning from imbalanced data: Progress and challenges, marking the 15-year anniversary," *J. Artif. Intell. Res.*, vol. 61, pp. 863–905, Apr. 2018.

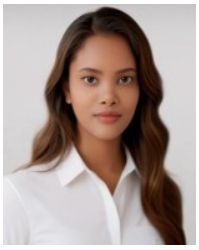
[31] D. Elreedy and A. F. Atiya, "A comprehensive analysis of synthetic minority oversampling technique (SMOTE) for handling class imbalance," *Inf. Sci.*, vol. 505, pp. 32–64, Dec. 2019.

[32] H. Ali, M. N. Mohd Salleh, R. Saedudin, K. Hussain, and M. F. Mushtaq, "Imbalance class problems in data mining: A review," *Indonesian J. Electr. Eng. Comput. Sci.*, vol. 14, no. 3, pp. 1552–1563, Jun. 2019.

[33] RIT Croatia. (2025). *Assessme Project Recognized Among Europe's Top Innovations in Higher Education*. Accessed: Jul. 18, 2025. [Online]. Available: https://www.rit.edu/croatia/news/assessme-project-recognized-among-europes-top-innovations-higher-education



ALAN MUTKA received the bachelor's and Ph.D. degrees from the Faculty of Electrical Engineering and Computing (FER). He is currently a Research Associate and a Senior Lecturer with RIT Croatia, where he is the Area Head of the Information Technology (Web & Mobile Computing) Program. He is an Active Member of the Laboratory for Robotics and Intelligent Control Systems (LARICS). He participated in numerous scientific and professional projects. He is the Co-founder of Cognitus D.O.O., a company focused on consulting and developing advanced software solutions. He is the author or co-author of more than 40 scientific papers published in prestigious international journals, conference proceedings, and books. He is actively involved as the Project Lead, a Researcher, and a Collaborator on numerous industrial and scientific projects in Croatia and across EU. His expertise and research interests include data visualization, robotics, computer-aided design (CAD), computer-aided manufacturing (CAM), deep learning, and big data analytics, with a particular focus on behavioral analysis in education and cybersecurity.



FATIMA ŽIVKOVIĆ MUTKA received the bachelor's degree in information technology from Rochester Institute of Technology. She is currently a Senior Consultant with Scheer PAS, Zagreb, Croatia, specializing in digital transformation, process automation, and frontend development. She has extensive experience in business process analysis and software development. She is fluent in several languages and passionate about helping organizations navigate digital change.



MARTIN ŽAGAR He is currently employed at RIT Croatia and member of European Commission's Observatory on Online Platforms and several sectoral associations. He Primarily enjoys applying scientific achievements in computer science to the real sector and working with students. He has published over 130 scientific and professional articles and has participated in various types of international and domestic, professional, and research projects, both as the Project Leader and a Researcher. His research interests include multimedia technologies, high-performance computing, data compression, computer architectures, digital platforms, and telemedicine.



DOMAGOJ TOLIĆ received the Ph.D. degree in control systems from the University of New Mexico, Albuquerque, NM, USA, in 2012. He is currently with RIT Croatia in Dubrovnik and the Laboratory for Intelligent Autonomous Systems (LARIAT). He was a Postdoctoral Researcher with the University of Zagreb, the Technical University of Munich, and the University of Dubrovnik. His professional research interests include stability and estimation of nonlinear networked control systems with applications in robotics and multi-agent systems.